

# Requirements & Feature Specification

Product definition, roles, modules, and functional/non-functional requirements

**Document status:** Active

**Version:** 1.0

**Last updated:** 2026-09-26

## 1. Product Definition

FleetFlow is a fleet and logistics management platform for managing vehicles, users, requests, operational workflows, service history, and fleet-related information through a role-based web application.

## 2. User Roles

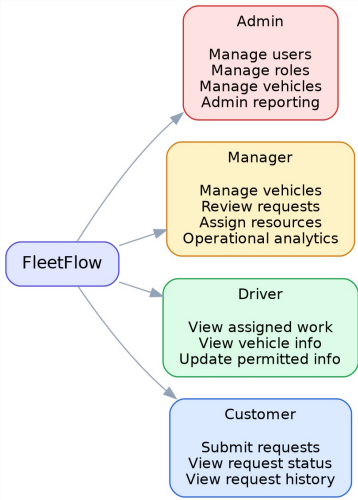


Figure 1 — FleetFlow role overview and primary responsibilities

### Admin

Admin has platform-level responsibilities.

Expected capabilities:

- Manage users
- Manage roles and permissions
- Manage vehicles
- View fleet information
- Review operational activity
- Access administrative reporting
- Configure platform-level settings

### Manager

Manager focuses on fleet operations.

Expected capabilities:

- View fleet
- Manage vehicles
- Review vehicle requests
- Assign/manage operational resources where authorized
- Review vehicle service/history
- Monitor fleet status
- Access operational analytics

### Driver

Driver focuses on assigned operational work.

Expected capabilities:

- View assigned vehicle/work
- View relevant vehicle information
- Update permitted operational information
- View relevant requests/tasks

### Customer

Customer interacts with FleetFlow primarily to request fleet resources/services.

Expected capabilities:

- View available/requestable information
- Submit vehicle requests
- View request status
- Review relevant request history

Exact permissions should be enforced by backend authorization, not only hidden in the frontend.

## 3. Core Modules

### 3.1 Authentication

Requirements:

- User registration
- User login
- Password validation
- JWT authentication
- Role-aware user state
- Protected application routes
- Logout

3.2 Dashboard

Requirements:

- Role-aware dashboard
- Fleet summary
- Relevant operational metrics
- Quick actions
- Recent activity
- Responsive layout

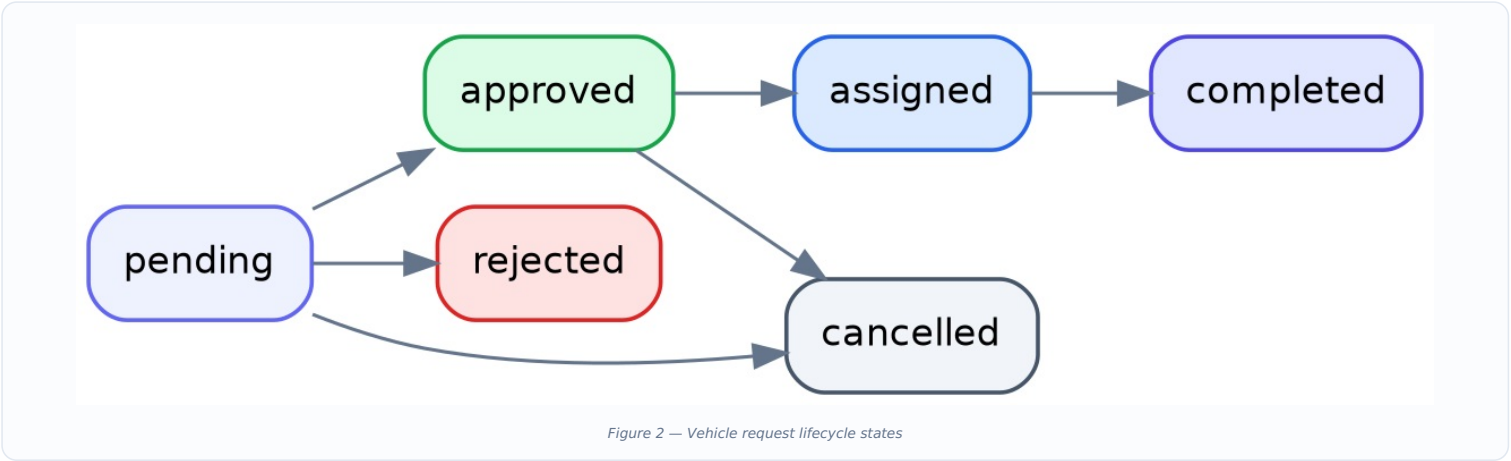
3.3 Vehicle Management

Requirements:

- Create vehicle
- View vehicle list
- Search/filter vehicles
- View vehicle details
- Update vehicle
- Deactivate vehicle
- Update vehicle status

Vehicle status should be represented by a controlled set of states rather than arbitrary strings.

3.4 Vehicle Requests



Requirements:

- Submit a vehicle request
- View request status
- View request details
- Review pending requests
- Support role-based request handling
- Maintain request history

3.5 Vehicle Service / History

Requirements:

- Record vehicle service information
- Display service history
- Display service dates clearly
- Show service details
- Associate records with vehicles
- Preserve historical records

3.6 Driver Management

Planned requirements:

- Driver profiles
- Driver status
- Vehicle assignment
- Driver-related operational history

3.7 Maintenance

- Planned requirements:
- Maintenance records
  - Maintenance schedules
  - Maintenance status
  - Cost tracking
  - Vehicle maintenance history
  - Upcoming maintenance indicators

3.8 Notifications

- Planned requirements:
- In-app notifications
  - Request status notifications
  - Assignment notifications
  - Maintenance notifications
  - Read/unread state

3.9 Analytics & Reporting

- Planned requirements:
- Fleet utilization
  - Vehicle status distribution
  - Maintenance metrics
  - Request metrics
  - Operational summaries
  - Date-based filtering

4. Functional Requirements

| ID     | Requirement                                          | Priority |
|--------|------------------------------------------------------|----------|
| FR-001 | Users can register                                   | High     |
| FR-002 | Users can log in                                     | High     |
| FR-003 | System identifies user role                          | High     |
| FR-004 | Protected resources require authentication           | High     |
| FR-005 | Authorized users can manage vehicles                 | High     |
| FR-006 | Users can view vehicle information                   | High     |
| FR-007 | Users can submit vehicle requests when permitted     | High     |
| FR-008 | Authorized users can review requests                 | High     |
| FR-009 | Vehicle service/history can be recorded              | High     |
| FR-010 | Vehicle history remains associated with its vehicle  | High     |
| FR-011 | Unauthorized users cannot perform restricted actions | High     |
| FR-012 | UI supports responsive layouts                       | Medium   |
| FR-013 | UI supports light/dark themes                        | Medium   |
| FR-014 | Fleet analytics can be introduced                    | Medium   |
| FR-015 | Notifications can be introduced                      | Medium   |

5. Non-Functional Requirements

Security

- Passwords must never be stored in plaintext.
- JWT secrets must come from environment configuration.
- Protected endpoints must verify authentication.
- Authorization must be enforced server-side.
- User input must be validated.

Performance

- API responses should remain lightweight.
- Lists should support pagination/filtering as data grows.
- Database queries should use appropriate indexes.
- Frontend should avoid unnecessary API requests.

Reliability

- Services should fail gracefully.
- API errors should use consistent response structures.
- Validation errors should be understandable.
- Important state changes should be auditable as the platform matures.

Maintainability

- TypeScript should be preferred.

- Business logic should remain separated from route handlers.
- Shared conventions should be documented.
- Services should have clear ownership boundaries.

## 6. Business Rules

1. A user must be authenticated before accessing protected resources.
2. A user's role determines which operations they are authorized to perform.
3. Frontend role checks are for user experience; backend checks are authoritative.
4. Deactivated vehicles should not be treated as normally available.
5. Vehicle requests should have controlled lifecycle states.
6. Historical service records should not be silently overwritten when a new service event is recorded.
7. Cross-service operations should not rely on direct database access.
8. Secrets must never be stored in source control.

## 7. Acceptance Principles

A feature is considered complete when:

- The UI supports the intended workflow.
- Backend validation exists.
- Backend authorization exists.
- Error states are handled.
- Loading states are handled.
- The feature works on desktop and mobile layouts where applicable.
- Data persists correctly.
- Relevant documentation is updated.

## 8. Future Product Areas

Potential future modules:

- Fuel management
- GPS/location tracking
- Driver performance
- Expense management
- Document management
- Automated maintenance reminders
- Advanced analytics
- Audit logs
- Real-time fleet monitoring